github.com/anvi23mth/inventory-system/internal/service/package_service.go (100.0%)
github.com/anvi23mth/inventory-system/internal/service/category_service.go (100.0%)
github.com/anvi23mth/inventory-system/internal/service/package_service.go (100.0%)

not tracked not covered covered

```
import (
	"context"
	"errors" // Required for validation errors

	"github.com/anvi23mth/inventory-system/internal/model"
	"github.com/anvi23mth/inventory-system/internal/repository"
)

// ProductService must start with a Capital 'P' to be exported
type ProductService struct {
	Repo repository.ProductRepository
}

// NewProductService initializes the service
func NewProductService(r repository.ProductRepository) *ProductService {
	return &ProductService{Repo: r}
}

func (s *ProductService) CreateProduct(ctx context.Context, p model.Product) (model.Product, error) {
	// --- WEEK 3 VALIDATION LOGIC ---
	if p.Price < 0 {
		return model.Product{}, errors.New("price cannot be negative")
	}
	if p.Name == "" {
		return model.Product{}, errors.New("product name is required")
	}
	// -------------------------------

	err := s.Repo.Create(ctx, p)
	return p, err
}
func (s *ProductService) ListProducts(ctx context.Context) ([]model.Product, error) {
	products, err := s.Repo.GetAll(ctx)
	if err != nil {
		return nil, err // This line is what your test is looking for!
	}
	return products, nil
}

func (s *ProductService) GetProductByID(ctx context.Context, id string) (model.Product, error) {
	product, err := s.Repo.GetByID(ctx, id)
	if err != nil {
		return model.Product{}, err // Returns the error to the caller
	}
	return product, nil
}

func (s *ProductService) DeleteProduct(ctx context.Context, id string) error {
	// We must capture the error from the repository to return it.
	// This allows our tests to verify both success and failure paths.
	err := s.Repo.Delete(ctx, id)
	return err
}
func (s *ProductService) UpdateProduct(ctx context.Context, id string, p model.Product) (model.Product, error) {
	// Week 5: Add validation to Update logic for 100% coverage
	if p.Price < 0 {
		return model.Product{}, errors.New("price cannot be negative")
	}
	if p.Name == "" {
		return model.Product{}, errors.New("product name is required")
	}
	err := s.Repo.Update(ctx, id, p)
	return p, err
}
```